Ablation : ConvNeXt-Tiny — SiLU | StochDepth | BatchNorm

Config: ImageNet- K pretrained | SiLU activation | StochDepth | BatchNorm **Model:** ConvNeXt

Dataset: Rice Leaf Disease Identification Dataset

. Colab Setup (jalankan sekali per session)

In [1]:
```
from google.colab import drive
drive.mount('/content/drive')
import os
DRIVE_DIR = '/content/drive/MyDrive/rice-convnext/06_SiLU_StochDepth_Batc
os.makedirs(DRIVE_DIR, exist_ok=True)
print(f'Drive folder ready: {DRIVE_DIR}')
```

```
Mounted at /content/drive
Drive folder ready: /content/drive/MyDrive/rice-convnext/
06_SiLU_StochDepth_BatchNorm
```

In [2]:
```
from google.colab import files
import os, shutil
print('Upload kaggle.json...')
files.upload()
os.makedirs('/root/.kaggle', exist_ok=True)
shutil.move('kaggle.json', '/root/.kaggle/kaggle.json')
os.chmod('/root/.kaggle/kaggle.json', 0o600)
print('Kaggle API ready!')
```

```
Upload kaggle.json...
```

Upload widget is only available when the cell has been executed in the current browser session. Pl rerun this cell to enable.

```
Saving kaggle.json to kaggle.json
Kaggle API ready!
```

In [3]:
```
DATASET_DIR = '/content/drive/MyDrive/rice-convnext/dataset'
if os.path.exists(DATASET_DIR) and len(os.listdir(DATASET_DIR)) > 0:
    print('Dataset sudah ada, skip download.')
else:
    !pip install kaggle -q
    !kaggle datasets download -d wangxiaoqii/rice-leaf-disease-identifica
    !unzip -q /content/tmp/*.zip -d {DATASET_DIR}
    !rm -rf /content/tmp
    print('Dataset ready!')

import shutil
LOCAL_DATASET = '/content/dataset'
if not os.path.exists(LOCAL_DATASET):
    print('Copying dataset ke local SSD...')
    shutil.copytree(DATASET_DIR, LOCAL_DATASET)
    print('Done!')
else:
```

```
    print('Sudah ada di local.')
DATA_DIR = LOCAL_DATASET
```

```
Dataset sudah ada, skip download.
Copying dataset ke local SSD...
Done!
```

. Setup & Imports

In [4]:
```
import os, random, shutil
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
from sklearn.metrics import classification_report, confusion_matrix
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms, models

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed(SEED)
torch.backends.cudnn.deterministic = True

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {DEVICE}')
if torch.cuda.is_available():
    print(f'GPU: {torch.cuda.get_device_name(0)}')
```

```
Device: cuda
GPU: Tesla T4
```

. Configuration

In [5]:
```
DATA_DIR      = '/content/dataset'
DRIVE_DIR     = '/content/drive/MyDrive/rice-convnext/06_SiLU_StochDepth_
IMG_SIZE      = 224
BATCH_SIZE    = 64
NUM_EPOCHS    = 30
LR_INIT       = 1e-3
LR_FINAL      = 1e-6
WEIGHT_DECAY  = 0.05
NUM_WORKERS   = 2
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD  = [0.229, 0.224, 0.225]
SAVE_PATH     = f'{DRIVE_DIR}/convnext_tiny_06_silu_stochdepth_batchnorm.
EXP_NAME      = 'ConvNeXt-Tiny | SiLU | StochDepth | BatchNorm'
print(f'Experiment: {EXP_NAME}')
```

```
Experiment: ConvNeXt-Tiny | SiLU | StochDepth | BatchNorm
```

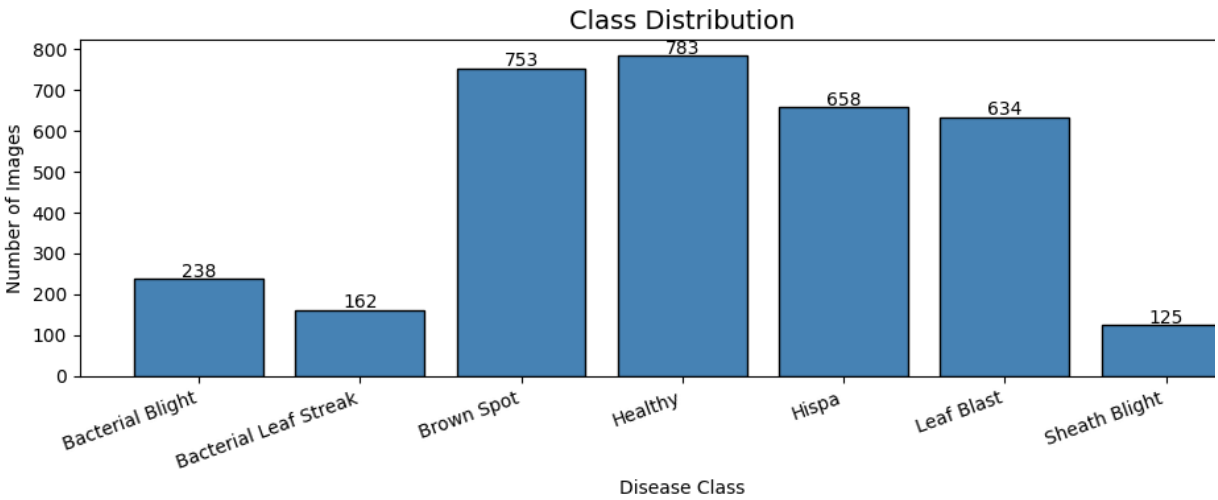
. Data Loading & Augmentation

In [6]:
```
train_transforms = transforms.Compose([
    transforms.RandomResizedCrop(IMG_SIZE),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(30),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
])
val_test_transforms = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(IMG_SIZE),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
])

full_dataset = datasets.ImageFolder(DATA_DIR)
CLASS_NAMES  = full_dataset.classes
NUM_CLASSES  = len(CLASS_NAMES)
print(f'Classes ({NUM_CLASSES}): {CLASS_NAMES}')
print(f'Total images: {len(full_dataset)}')

total   = len(full_dataset)
n_train = int(0.6 * total)
n_val   = int(0.2 * total)
n_test  = total - n_train - n_val

generator = torch.Generator().manual_seed(SEED)
train_set, val_set, test_set = random_split(
    full_dataset, [n_train, n_val, n_test], generator=generator
)

train_set.dataset.transform = train_transforms
val_set.dataset.transform   = val_test_transforms
test_set.dataset.transform  = val_test_transforms

train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True,
val_loader   = DataLoader(val_set,   batch_size=BATCH_SIZE, shuffle=False
test_loader  = DataLoader(test_set,  batch_size=BATCH_SIZE, shuffle=False
print(f'Train: {len(train_set)} | Val: {len(val_set)} | Test: {len(test_s
```

```
Classes (7): ['Bacterial Blight', 'Bacterial Leaf Streak', 'Brown Spot', 'Hea
'Hispa', 'Leaf Blast', 'Sheath Blight']
Total images: 3353
Train: 2011 | Val: 670 | Test: 672
```

In [7]:
```
labels_all = [full_dataset.targets[i] for i in range(len(full_dataset))]
counts = [Counter(labels_all)[i] for i in range(NUM_CLASSES)]
plt.figure(figsize=(10, 4))
bars = plt.bar(CLASS_NAMES, counts, color='steelblue', edgecolor='black')
plt.title('Class Distribution', fontsize=14)
plt.xlabel('Disease Class'); plt.ylabel('Number of Images')
plt.xticks(rotation=20, ha='right')
for bar, count in zip(bars, counts):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height()+5, str(cou
plt.tight_layout()
```

```
plt.savefig('class_distribution.png', dpi=150)
plt.show()
```



. Model — ConvNeXt-Tiny (SiLU | StochDepth | BatchNorm)

In [8]:
```
class BatchNormNHWC(nn.Module):
    """BatchNorm2d wrapper that accepts NHWC input (used inside ConvNeXt
    def __init__(self, num_features):
        super().__init__()
        self.bn = nn.BatchNorm2d(num_features)
    def forward(self, x):  # x: (B, H, W, C)
        return self.bn(x.permute(0, 3, 1, 2)).permute(0, 2, 3, 1)

def replace_layernorm_with_batchnorm(module):
    """Recursively replace all LayerNorm variants with BatchNorm."""
    for name, child in module.named_children():
        if type(child).__name__ == 'LayerNorm2d':
            # Stem / downsampling: NCHW input → use BatchNorm2d
            C = child.normalized_shape[0]
            setattr(module, name, nn.BatchNorm2d(C))
        elif isinstance(child, nn.LayerNorm):
            # CNBlock (NHWC after Permute) → use BatchNormNHWC
            C = child.normalized_shape[0]
            setattr(module, name, BatchNormNHWC(C))
        else:
            replace_layernorm_with_batchnorm(child)

def replace_activation(model, src_cls, dst_fn):
    for name, module in model.named_children():
        if isinstance(module, src_cls):
            setattr(model, name, dst_fn())
        else:
            replace_activation(module, src_cls, dst_fn)

model = models.convnext_tiny(weights=models.ConvNeXt_Tiny_Weights.IMAGENE
in_features = model.classifier[2].in_features
model.classifier[2] = nn.Linear(in_features, NUM_CLASSES)

# Replace GELU → SiLU
replace_activation(model, nn.GELU, nn.SiLU)
print('Activation: GELU → SiLU')
```

```
# Replace LayerNorm → BatchNorm
replace_layernorm_with_batchnorm(model)
# Fix classifier: after Flatten input is (B, C), use BatchNorm1d
if isinstance(model.classifier[2], BatchNormNHWC):
    C = model.classifier[2].bn.num_features
    model.classifier[2] = nn.BatchNorm1d(C)
print('Normalization: LayerNorm → BatchNorm')

model = model.to(DEVICE)
total_p     = sum(p.numel() for p in model.parameters())
trainable_p = sum(p.numel() for p in model.parameters() if p.requires_gra
print(f'Experiment : {EXP_NAME}')
print(f'Total params    : {total_p:,}')
print(f'Trainable params: {trainable_p:,}')
```

```
Downloading: "https://download.pytorch.org/models/convnext_tiny-983f1562.pth"
root/.cache/torch/hub/checkpoints/convnext_tiny-983f1562.pth
100%|██████████| 109M/109M [00:00<00:00, 120MB/s]
Activation: GELU → SiLU
Normalization: LayerNorm → BatchNorm
Experiment : ConvNeXt-Tiny | SiLU | StochDepth | BatchNorm
Total params    : 27,825,511
Trainable params: 27,825,511
```

. Training Setup

In [9]:

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=LR_INIT, weight_decay=WEIG

def get_scheduler(optimizer, num_epochs, lr_init, lr_final):
    warmup = optim.lr_scheduler.LinearLR(optimizer, start_factor=0.01, en
    cosine = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=num_ep
    return optim.lr_scheduler.SequentialLR(optimizer, schedulers=[warmup,

scheduler = get_scheduler(optimizer, NUM_EPOCHS, LR_INIT, LR_FINAL)
```

. Training Loop

In [10]:

```
def train_one_epoch(model, loader, criterion, optimizer):
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * images.size(0)
        _, predicted = outputs.max(1)
        correct += predicted.eq(labels).sum().item()
        total   += labels.size(0)
    return running_loss / total, correct / total

def evaluate(model, loader, criterion):
    model.eval()
```

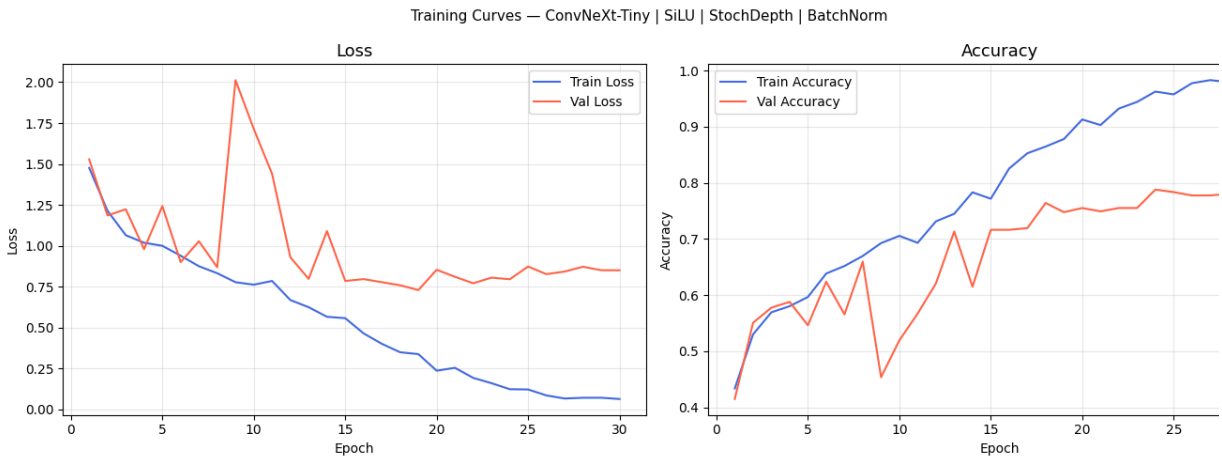
```
    running_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            outputs = model(images)
            loss = criterion(outputs, labels)
            running_loss += loss.item() * images.size(0)
            _, predicted = outputs.max(1)
            correct += predicted.eq(labels).sum().item()
            total   += labels.size(0)
    return running_loss / total, correct / total

history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc':
best_val_acc = 0.0

for epoch in range(NUM_EPOCHS):
    train_loss, train_acc = train_one_epoch(model, train_loader, criterio
    val_loss,   val_acc   = evaluate(model, val_loader, criterion)
    scheduler.step()
    history['train_loss'].append(train_loss)
    history['train_acc'].append(train_acc)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), SAVE_PATH)
    print(f'Epoch [{epoch+1:02d}/{NUM_EPOCHS}] '
          f'Train Loss: {train_loss:.4f} Acc: {train_acc:.4f} | '
          f'Val Loss: {val_loss:.4f} Acc: {val_acc:.4f}'
          + (' <- best' if val_acc == best_val_acc else ''))

print(f'\nBest Val Accuracy: {best_val_acc:.4f}')
print(f'Saved: {SAVE_PATH}')
```

```
Epoch [01/30] Train Loss: 1.4760 Acc: 0.4336 | Val Loss: 1.5283 Acc: 0.4149 <
Epoch [02/30] Train Loss: 1.2141 Acc: 0.5301 | Val Loss: 1.1851 Acc: 0.5507 <
Epoch [03/30] Train Loss: 1.0645 Acc: 0.5694 | Val Loss: 1.2233 Acc: 0.5776 <
Epoch [04/30] Train Loss: 1.0184 Acc: 0.5803 | Val Loss: 0.9794 Acc: 0.5881 <
Epoch [05/30] Train Loss: 0.9999 Acc: 0.5967 | Val Loss: 1.2425 Acc: 0.5463
Epoch [06/30] Train Loss: 0.9391 Acc: 0.6385 | Val Loss: 0.9001 Acc: 0.6239 <
Epoch [07/30] Train Loss: 0.8749 Acc: 0.6519 | Val Loss: 1.0281 Acc: 0.5657
Epoch [08/30] Train Loss: 0.8322 Acc: 0.6698 | Val Loss: 0.8685 Acc: 0.6597 <
Epoch [09/30] Train Loss: 0.7772 Acc: 0.6927 | Val Loss: 2.0125 Acc: 0.4537
Epoch [10/30] Train Loss: 0.7622 Acc: 0.7056 | Val Loss: 1.7138 Acc: 0.5194
Epoch [11/30] Train Loss: 0.7849 Acc: 0.6932 | Val Loss: 1.4392 Acc: 0.5672
Epoch [12/30] Train Loss: 0.6681 Acc: 0.7315 | Val Loss: 0.9296 Acc: 0.6209
Epoch [13/30] Train Loss: 0.6245 Acc: 0.7449 | Val Loss: 0.7977 Acc: 0.7134 <
Epoch [14/30] Train Loss: 0.5659 Acc: 0.7832 | Val Loss: 1.0900 Acc: 0.6149
Epoch [15/30] Train Loss: 0.5574 Acc: 0.7718 | Val Loss: 0.7848 Acc: 0.7164 <
Epoch [16/30] Train Loss: 0.4648 Acc: 0.8255 | Val Loss: 0.7962 Acc: 0.7164 <
Epoch [17/30] Train Loss: 0.4008 Acc: 0.8528 | Val Loss: 0.7774 Acc: 0.7194 <
Epoch [18/30] Train Loss: 0.3492 Acc: 0.8647 | Val Loss: 0.7586 Acc: 0.7642 <
Epoch [19/30] Train Loss: 0.3376 Acc: 0.8782 | Val Loss: 0.7293 Acc: 0.7478
Epoch [20/30] Train Loss: 0.2364 Acc: 0.9130 | Val Loss: 0.8533 Acc: 0.7552
Epoch [21/30] Train Loss: 0.2541 Acc: 0.9030 | Val Loss: 0.8103 Acc: 0.7493
Epoch [22/30] Train Loss: 0.1923 Acc: 0.9324 | Val Loss: 0.7707 Acc: 0.7552
Epoch [23/30] Train Loss: 0.1600 Acc: 0.9443 | Val Loss: 0.8052 Acc: 0.7552
Epoch [24/30] Train Loss: 0.1232 Acc: 0.9627 | Val Loss: 0.7953 Acc: 0.7881 <
Epoch [25/30] Train Loss: 0.1217 Acc: 0.9577 | Val Loss: 0.8731 Acc: 0.7836
Epoch [26/30] Train Loss: 0.0852 Acc: 0.9776 | Val Loss: 0.8265 Acc: 0.7776
Epoch [27/30] Train Loss: 0.0664 Acc: 0.9831 | Val Loss: 0.8426 Acc: 0.7776
Epoch [28/30] Train Loss: 0.0709 Acc: 0.9796 | Val Loss: 0.8716 Acc: 0.7806
Epoch [29/30] Train Loss: 0.0711 Acc: 0.9791 | Val Loss: 0.8507 Acc: 0.7821
Epoch [30/30] Train Loss: 0.0634 Acc: 0.9816 | Val Loss: 0.8501 Acc: 0.7791

Best Val Accuracy: 0.7881
Saved: /content/drive/MyDrive/rice-convnext/06_SiLU_StochDepth_BatchNorm/
convnext_tiny_06_silu_stochdepth_batchnorm.pt
```

. Training Curves

In [11]:

```
epochs_range = range(1, NUM_EPOCHS + 1)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(epochs_range, history['train_loss'], label='Train Loss', col
axes[0].plot(epochs_range, history['val_loss'],   label='Val Loss',   col
axes[0].set_title('Loss', fontsize=13)
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Loss')
axes[0].legend(); axes[0].grid(alpha=0.3)
axes[1].plot(epochs_range, history['train_acc'], label='Train Accuracy',
axes[1].plot(epochs_range, history['val_acc'],   label='Val Accuracy',
axes[1].set_title('Accuracy', fontsize=13)
axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Accuracy')
axes[1].legend(); axes[1].grid(alpha=0.3)
plt.suptitle(f'Training Curves — {EXP_NAME}', fontsize=11)
plt.tight_layout()
plt.savefig('training_curves.png', dpi=150)
plt.show()
```



. Evaluation on Test Set

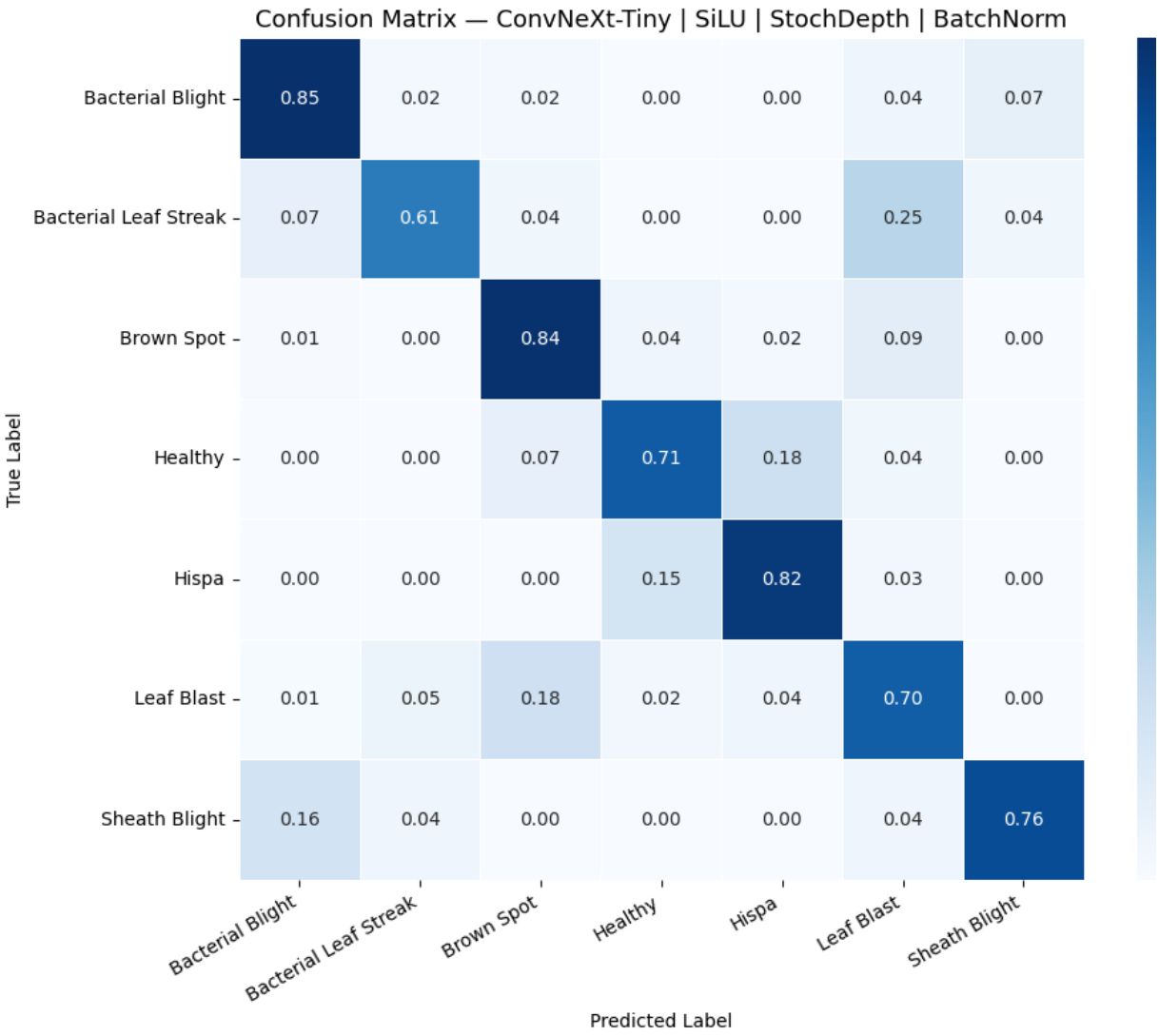
In [12]:
```
model.load_state_dict(torch.load(SAVE_PATH, map_location=DEVICE))
model.eval()
all_preds, all_labels = [], []
with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(DEVICE)
        outputs = model(images)
        _, predicted = outputs.max(1)
        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.numpy())

report = classification_report(all_labels, all_preds, target_names=CLASS_
report_df = pd.DataFrame(report).transpose()
print('=== Classification Report ===')
print(report_df.round(4).to_string())
report_df.to_csv('classification_report.csv')

summary = {
    'Model'     : ['ConvNeXt-Tiny'],
    'Activation': ['SiLU'],
    'Dropout'   : ['StochDepth'],
    'Norm'      : ['BatchNorm'],
    'Accuracy'  : [report['accuracy']],
    'Precision' : [report['weighted avg']['precision']],
    'Recall'    : [report['weighted avg']['recall']],
    'F1-Score'  : [report['weighted avg']['f1-score']],
}
summary_df = pd.DataFrame(summary)
print('\n=== Summary ===')
print(summary_df.round(4).to_string(index=False))
summary_df.to_csv('summary.csv', index=False)
```

```
=== Classification Report ===
                      precision   recall  f1-score   support
Bacterial Blight         0.8519  0.8519    0.8519   54.0000
Bacterial Leaf Streak    0.6800  0.6071    0.6415   28.0000
Brown Spot               0.7955  0.8434    0.8187  166.0000
Healthy                  0.8138  0.7108    0.7588  166.0000
Hispa                    0.7031  0.8182    0.7563  110.0000
Leaf Blast               0.7167  0.6992    0.7078  123.0000
Sheath Blight            0.7917  0.7600    0.7755   25.0000
accuracy                 0.7679  0.7679    0.7679    0.7679
macro avg                0.7647  0.7558    0.7586  672.0000
weighted avg             0.7700  0.7679    0.7671  672.0000

=== Summary ===
        Model Activation    Dropout      Norm  Accuracy  Precision  Recall  F
Score
ConvNeXt-Tiny       SiLU StochDepth BatchNorm    0.7679       0.77  0.7679
0.7671
```

. Confusion Matrix

In [13]:

```python
cm = confusion_matrix(all_labels, all_preds)
cm_norm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
plt.figure(figsize=(10, 8))
sns.heatmap(cm_norm, annot=True, fmt='.2f',
            xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES,
            cmap='Blues', linewidths=0.5)
plt.title(f'Confusion Matrix — ConvNeXt-Tiny | SiLU | StochDepth | BatchN
plt.ylabel('True Label'); plt.xlabel('Predicted Label')
plt.xticks(rotation=30, ha='right')
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150)
plt.show()
```



. Save All to Drive

In [14]:
```
local_files = ['training_curves.png', 'confusion_matrix.png',
               'classification_report.csv', 'summary.csv', 'class_distrib
for fname in local_files:
    if os.path.exists(fname):
        shutil.copy(fname, f'{DRIVE_DIR}/{fname}')

all_outputs = [
    (f'{DRIVE_DIR}/convnext_tiny_06_silu_stochdepth_batchnorm.pt',
    (f'{DRIVE_DIR}/training_curves.png',              'Training curves'),
    (f'{DRIVE_DIR}/confusion_matrix.png',             'Confusion matrix')
    (f'{DRIVE_DIR}/classification_report.csv',        'Per-class metrics'
    (f'{DRIVE_DIR}/summary.csv',                      'Summary table'),
]
print(f'=== Output Files: {DRIVE_DIR} ===')
for fpath, desc in all_outputs:
    exists = '✅' if os.path.exists(fpath) else '❌'
    print(f'{exists}  {os.path.basename(fpath):<45} {desc}')
```

```
=== Output Files: /content/drive/MyDrive/rice-convnext/06_SiLU_StochDepth_Bat
===
✅  convnext_tiny_06_silu_stochdepth_batchnorm.pt Model weights
✅  training_curves.png                           Training curves
✅  confusion_matrix.png                          Confusion matrix
✅  classification_report.csv                     Per-class metrics
✅  summary.csv                                   Summary table
```